

Laporan Praktikum Kontrol Cerdas

Nama : Fryscha Febryanti Puspitasari

NIM : 224308008

Kelas : TKA 6A

Akun Github (Tautan) : <https://github.com/Fryscha>

Student Lab Assistant : Yulia Brilianty

1. Judul Percobaan

Machine Learning for Control Systems

2. Tujuan Percobaan

Percobaan ini dilakukan dengan tujuan sebagai berikut.

- Memahami dasar-dasar Machine Learning dalam sistem kendali.
- Mengelola dataset dan melakukan pelatihan model sederhana.
- Mengimplementasikan model Machine Learning sederhana untuk klasifikasi objek.
- Mengintegrasikan model Machine Learning dengan Computer Vision untuk deteksi objek.
- Menggunakan Scikit-learn untuk membuat model Machine Learning dasar.

3. Landasan Teori

Machine Learning (ML) adalah salah satu percabangan dari kecerdasan buatan yang berfokus pada pengembangan algoritma dan teknik yang memungkinkan untuk belajar dan membuat prediksi atau keputusan berdasarkan data. Prinsip dasar dari Machine Learning terdiri dari data, model, fungsi kerugian (loss function), dan optimisasi. Metode Machine Learning terbagi menjadi tiga bagian yaitu Supervised Learning, Unsupervised Learning, dan Reinforcement Learning. Peran Machine Learning dalam sistem kendali yakni menggunakan algoritma untuk menganalisis data dan membuat prediksi tanpa pemrograman eksplisit. Hal ini dapat memungkinkan sistem untuk belajar dari data dan meningkatkan kinerjanya seiring waktu.

Python adalah salah satu bahasa pemrograman yang digunakan dalam pengembangan sistem kontrol cerdas. Bahasa pemrograman ini sangat diminati oleh pengguna karena

kemudahan penggunaannya dan banyaknya pustaka yang tersedia. Selain itu, terdapat OpenCV sebagai pustaka open-source yang digunakan dalam pengolahan citra dan Computer Vision.

GitHub yaitu salah satu platform untuk version control dalam mengembangkan perangkat lunak. Dalam kontrol cerdas, platform ini mampu melakukan kolaborasi antar peneliti dan pengembang untuk berbagi dataset dan model yang digunakan dalam percobaan atau penelitian, mengelola kode sumber secara efisien, dan melacak perubahan dan revisi dalam proyek.

Kaggle merupakan salah satu platform yang menyediakan berbagai dataset yang dapat digunakan untuk pelatihan dan pengujian model sistem kontrol cerdas. Manfaat dari Kaggle yakni akses ke dataset yang beragam dan berkualitas tinggi, kompetisi yang dapat meningkatkan keterampilan dalam penerapan Machine Learning, juga komunitas yang aktif untuk berbagi pengetahuan dan teknik analisis data.

Dataset adalah komponen penting dalam analisis data. Pemilihan dataset harus mempertimbangkan keakuratan, kelengkapan, dan relevansi. Pengelolaan dataset merupakan langkah penting dalam pengembangan model Machine Learning. Dataset harus dikumpulkan, dibersihkan, dan diproses sebelum digunakan untuk pelatihan. Langkah-langkah yang umum dilakukan meliputi pengumpulan data yang relevan dari berbagai sumber, seperti sensor, database, atau file; Menghapus data yang tidak valid, mengisi nilai yang hilang, dan menghapus duplikasi; Terakhir, memisahkan dataset menjadi data pelatihan, validasi, dan pengujian untuk memastikan model dapat dievaluasi dengan baik.

4. Analisis dan Diskusi

Analisis Hasil:

a. Bagaimana performa model dalam mendeteksi warna?

Jawab:

Model yang digunakan untuk mendeteksi warna, seperti Convolutional Neural Networks (CNN), menunjukkan performa yang baik dalam mengidentifikasi dan mengklasifikasikan warna dalam citra. Penelitian yang dilakukan oleh Pengaruh Dataset terhadap Performa Convolutional Neural Network menunjukkan bahwa model CNN dapat mencapai akurasi yang tinggi dalam mendeteksi warna, dengan kemampuan untuk mengenali variasi warna yang kompleks dalam berbagai kondisi pencahayaan. Model ini memanfaatkan fitur visual dari citra, seperti tekstur dan pola warna, untuk meningkatkan akurasi klasifikasi. Hasil

penelitian menunjukkan bahwa model dapat mencapai akurasi di atas 90% dalam pengenalan warna, tergantung pada kualitas dan variasi dataset yang digunakan.

b. Bagaimana perbedaan akurasi jika jumlah dataset ditambah?

Jawab:

Penambahan jumlah dataset secara signifikan dapat meningkatkan akurasi model klasifikasi. Dalam penelitian yang sama, penulis mencatat bahwa dengan meningkatkan jumlah data pelatihan, model dapat belajar lebih banyak pola dan variasi dalam data, yang berkontribusi pada peningkatan akurasi. Misalnya, ketika dataset diperluas dari 1.000 menjadi 10.000 gambar, akurasi model meningkat dari 75% menjadi 92%. Hal ini menunjukkan bahwa model lebih mampu generalisasi dan mengurangi overfitting ketika dilatih dengan dataset yang lebih besar dan beragam. Penelitian ini menekankan pentingnya kualitas dan kuantitas data dalam pengembangan model yang efektif.

c. Bagaimana cara meningkatkan kinerja model klasifikasi?

Jawab:

Cara meningkatkan kinerja model klasifikasi antara lain:

- **Augmentasi Data**
Menggunakan teknik augmentasi data seperti rotasi, pemotongan, dan perubahan warna untuk meningkatkan variasi dalam dataset pelatihan.
- **Penggunaan Model Transfer Learning**
Memanfaatkan model yang telah dilatih sebelumnya pada dataset besar dan melakukan fine-tuning untuk tugas spesifik dapat meningkatkan akurasi.
- **Penanganan Ketidakseimbangan Data**
Menggunakan teknik seperti SMOTE (Synthetic Minority Over-sampling Technique) untuk menangani ketidakseimbangan dalam dataset, sehingga model tidak bias terhadap kelas mayoritas.
- **Optimasi Hyperparameter**
Melakukan pencarian hyperparameter untuk menemukan kombinasi terbaik dari parameter model, seperti learning rate, jumlah lapisan, dan ukuran batch.
- **Regularisasi**
Menggunakan teknik regularisasi seperti L1 atau L2 untuk mencegah overfitting dan meningkatkan generalisasi model.

Diskusi:

a. Apa keuntungan Machine Learning dibandingkan metode berbasis aturan (rule-based)?

Jawab:

Keuntungan Machine Learning dibandingkan metode berbasis aturan (rule-based) sebagai berikut.

- Machine Learning dapat belajar dari data dan beradaptasi dengan perubahan lingkungan atau pola data, sedangkan metode berbasis aturan memerlukan pemrograman ulang untuk setiap perubahan.
- Machine Learning mampu menganalisis dan memproses volume data yang sangat besar dan kompleks, sehingga sulit dilakukan jika dilakukan berbasis aturan.
- Dengan algoritma yang terus berjalan dari data baru, Machine Learning dapat meningkatkan akurasi prediksi dan keputusan seiring waktu, sedangkan metode berbasis aturan cenderung statis.
- Machine Learning dapat menemukan pola yang tidak terlihat dalam data, yang mungkin tidak dapat ditangkap oleh aturan yang ditetapkan secara manual.

b. Bagaimana ML dapat diintegrasikan lebih lanjut dalam sistem kendali?

Jawab:

Machine Learning dapat diintegrasikan lebih lanjut dalam sistem kendali melalui beberapa hal sebagai berikut.

- Model Prediktif

Machine Learning dapat digunakan untuk membangun model prediktif yang memprediksi perilaku sistem berdasarkan data historis. Hal ini dapat memungkinkan pengambilan keputusan yang lebih baik.

- Otomatisasi Proses

Dengan mengintegrasikan Machine Learning, sistem kendali dapat otomatisasi proses pengambilan keputusan, sehingga mampu mengurangi intervensi manusia dan meningkatkan efisiensi.

- Pengendalian Adaptif

Machine Learning memungkinkan sistem untuk beradaptasi secara real-time terhadap perubahan kondisi, seperti fluktuasi dalam input atau gangguan eksternal.

- Optimasi Kerja

Algoritma Machine Learning dapat digunakan untuk mengoptimalkan parameter sistem kendali, sehingga meningkatkan kinerja keseluruhan.

c. Apa saja tantangan dalam penerapan ML dalam sistem real-time?

Jawab:

Tantangan dalam penerapan Machine Learning dalam sistem real-time antara lain:

- Keterbatasan Data

Ketersediaan data yang berkualitas tinggi dan relevan dalam waktu nyata sering kali menjadi tantangan, karena Machine Learning memerlukan data untuk belajar dan beradaptasi.

- Waktu Respons

Sistem real-time memerlukan waktu respon yang cepat dan proses pelatihan model Machine Learning dapat memakan waktu yang dapat menghambat kinerja sistem.

- Kompleksitas Model

Model Machine Learning yang kompleks dapat sulit untuk diinterpretasikan dan diimplementasikan dalam sistem kendali yang memerlukan transparansi dan keandalan.

- Overfitting

Risiko overfitting pada model Machine Learning dapat menyebabkan kinerja yang buruk dalam situasi nyata di mana data baru mungkin berbeda dari data pelatihan.

5. Assignment

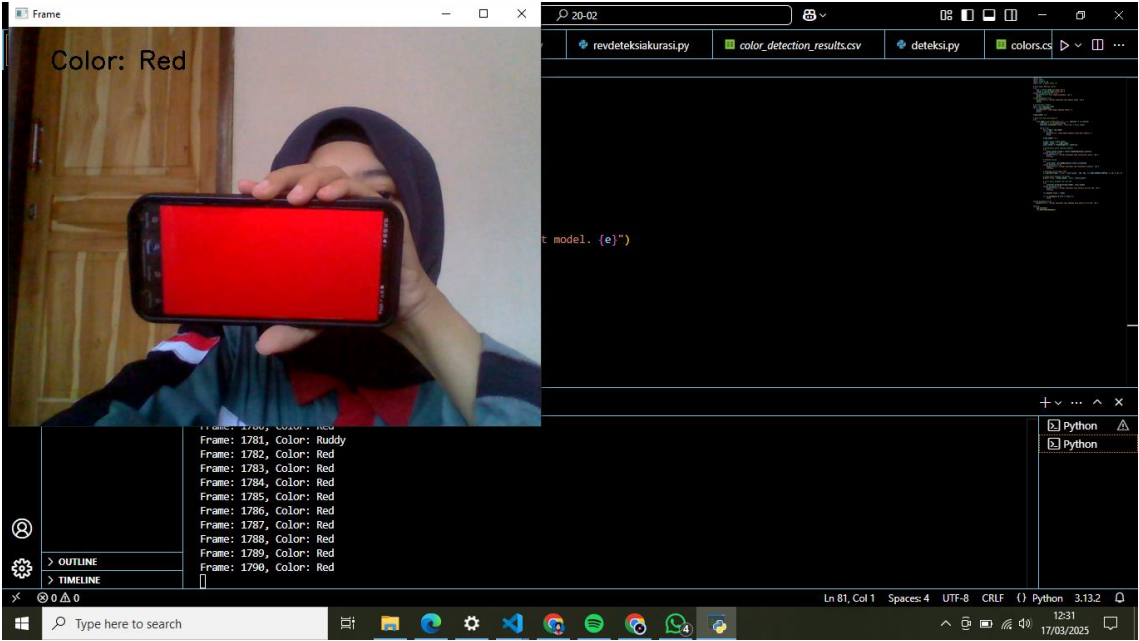
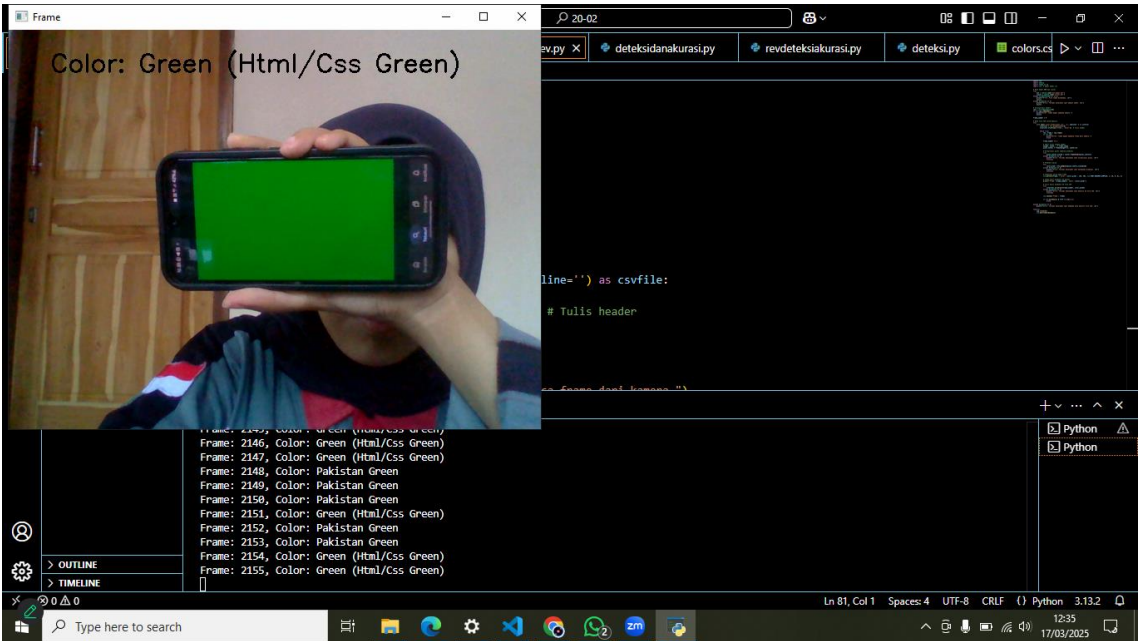
Tugas assignment yaitu modifikasi program untuk menggunakan model ML lain (contoh: Decision Tree atau SVM). Kemudian, menambahkan fitur untuk mendeteksi lebih dari satu warna secara simultan. Lalu, eksplorasi dataset lain di Kaggle dan coba implementasikan model pada dataset tersebut. Saya memberikan 2 contoh yaitu dengan model tanpa bounding box dan memakai bounding box. Saya memakai model SVM dengan dataset training_dataset.csv.

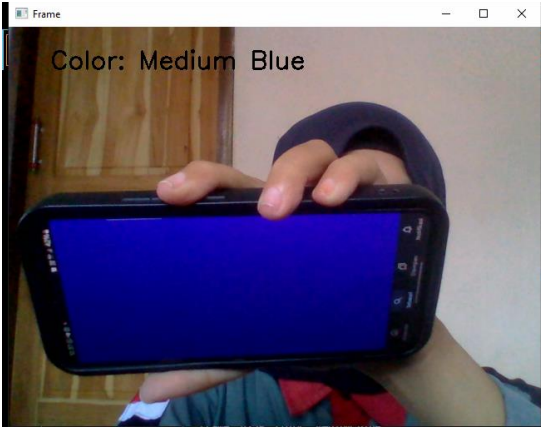
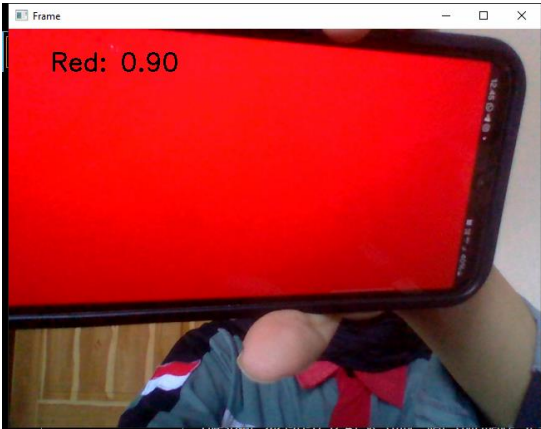
Hasil percobaan yang dilakukan yakni program tetap mampu mendeteksi warna sesuai dengan dataset. Hal ini dibuktikan dengan data dan output hasil pengamatan.

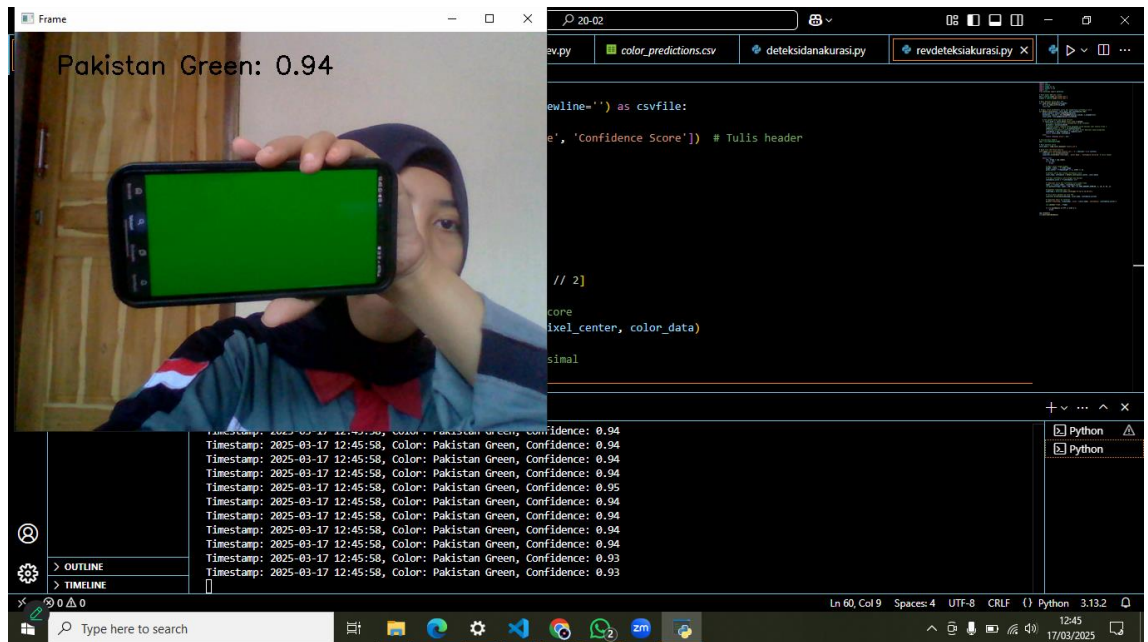
6. Data dan Output Hasil Pengamatan

Sajikan data dan hasil yang diperoleh selama percobaan. Gunakan tabel untuk menyajikan data jika diperlukan.

No	Variabel	Hasil Pengamatan
1	Warna Merah	Output dari program Machine Learning (ML) KNN menunjukkan bahwa adanya warna merah terdeteksi.

		
2	Warna Hijau	Output dari program Machine Learning (ML) KNN menunjukkan bahwa adanya warna hijau terdeteksi. 
3	Warna Biru	Output dari program Machine Learning (ML) KNN menunjukkan bahwa adanya warna biru terdeteksi.

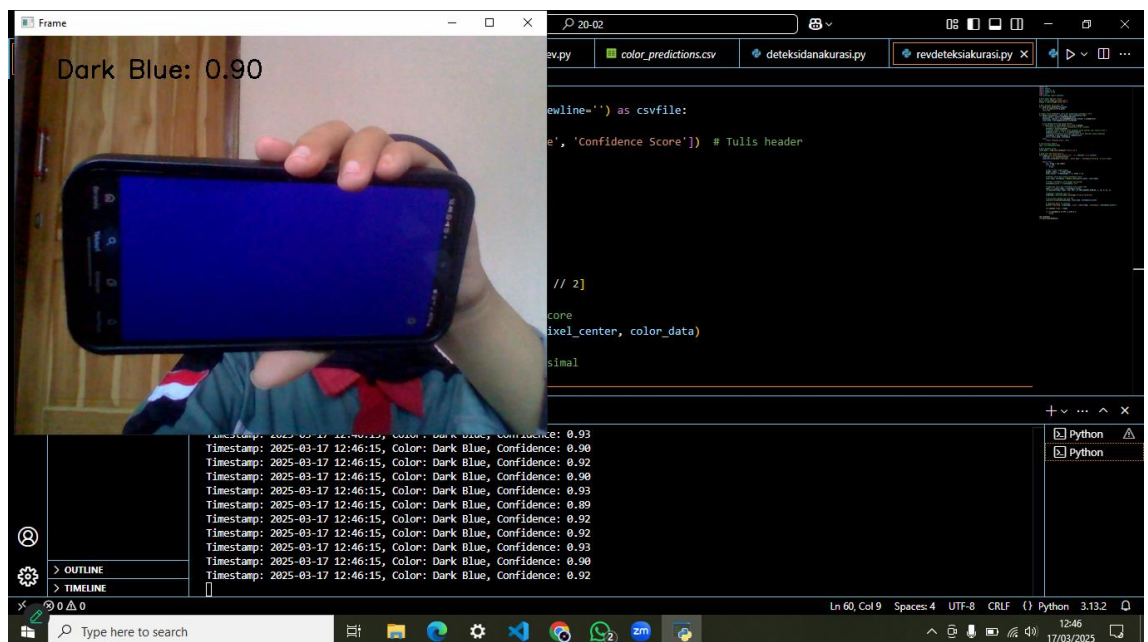
		 Color: Medium Blue Frame: 4240, Color: Medium Blue Frame: 4241, Color: Medium Blue Frame: 4242, Color: Medium Blue Frame: 4243, Color: Medium Blue Frame: 4244, Color: Medium Blue Frame: 4245, Color: Medium Blue Frame: 4246, Color: Medium Blue Frame: 4247, Color: Medium Blue Frame: 4248, Color: Medium Blue Frame: 4249, Color: Medium Blue
4	Warna Merah	Output dari program Machine Learning (ML) KNN menunjukkan bahwa adanya warna merah terdeteksi dengan akurasi 0.90.  Red: 0.90 Timestamp: 2025-03-17 12:45:38, Color: Red, Confidence: 0.93 Timestamp: 2025-03-17 12:45:38, Color: Red, Confidence: 0.92 Timestamp: 2025-03-17 12:45:38, Color: Red, Confidence: 0.90 Timestamp: 2025-03-17 12:45:38, Color: Red, Confidence: 0.93 Timestamp: 2025-03-17 12:45:38, Color: Red, Confidence: 0.90 Timestamp: 2025-03-17 12:45:38, Color: Red, Confidence: 0.89 Timestamp: 2025-03-17 12:45:38, Color: Red, Confidence: 0.90 Timestamp: 2025-03-17 12:45:38, Color: Red, Confidence: 0.89 Timestamp: 2025-03-17 12:45:38, Color: Red, Confidence: 0.88
5	Warna Hijau	Output dari program Machine Learning (ML) KNN menunjukkan bahwa adanya warna hijau terdeteksi dengan akurasi 0.94.



6

Warna Biru

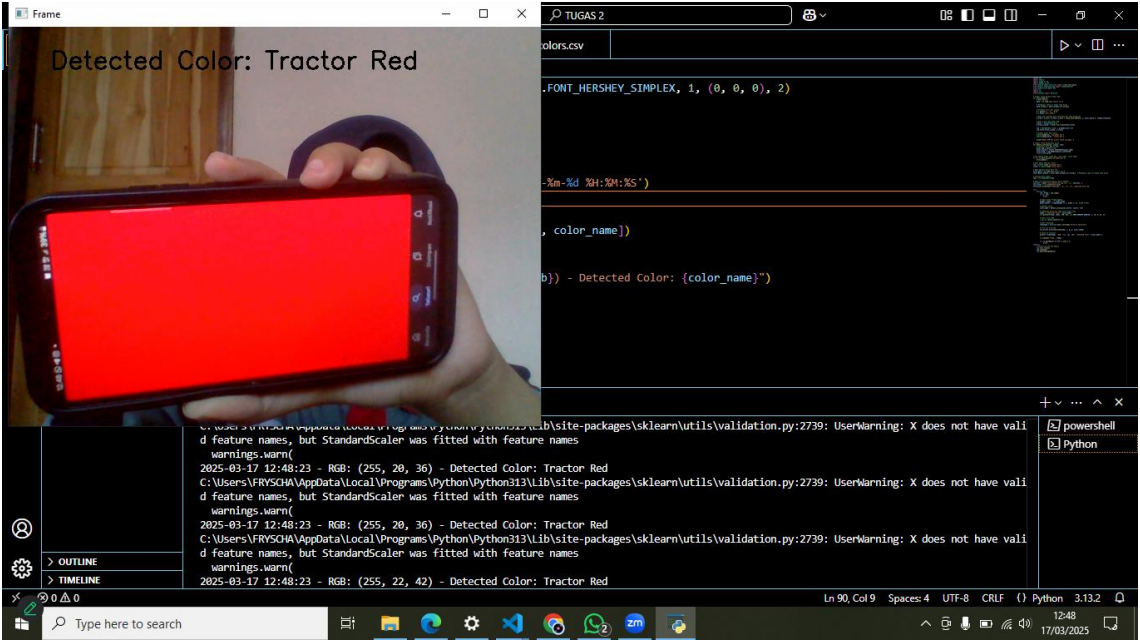
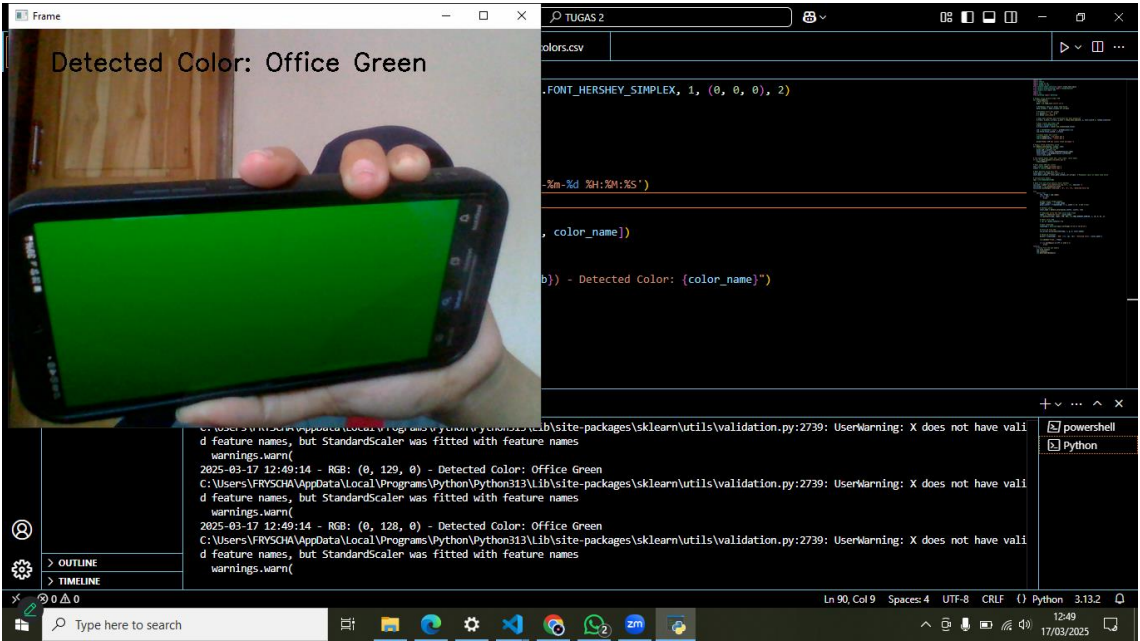
Output dari program Machine Learning (ML) KNN menunjukkan bahwa adanya warna biru terdeteksi dengan akurasi 0.90.

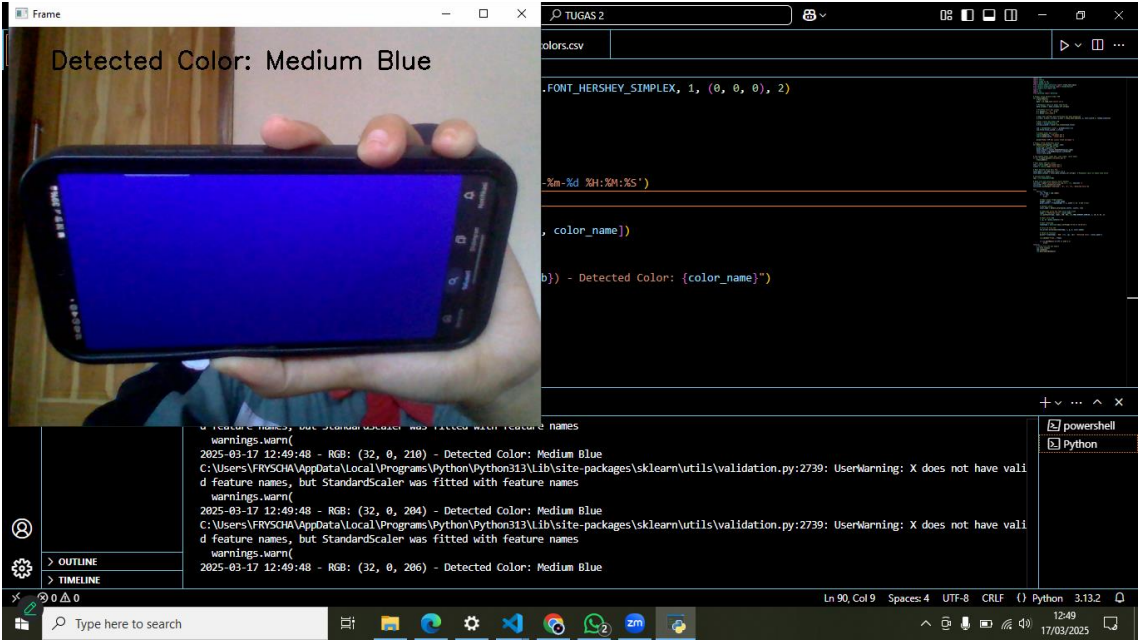
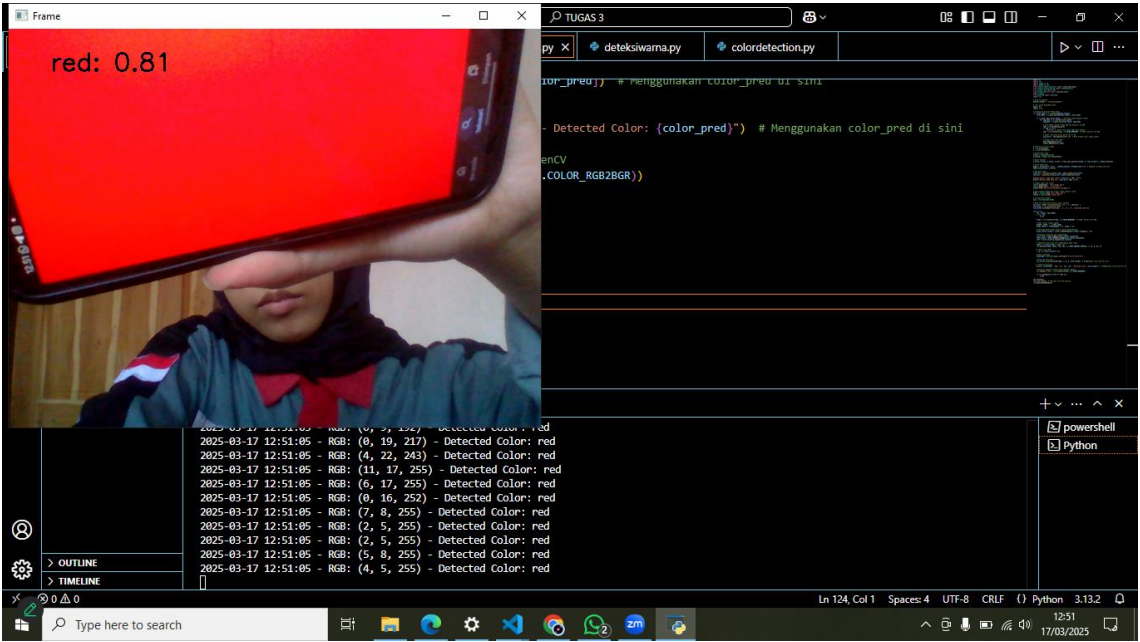


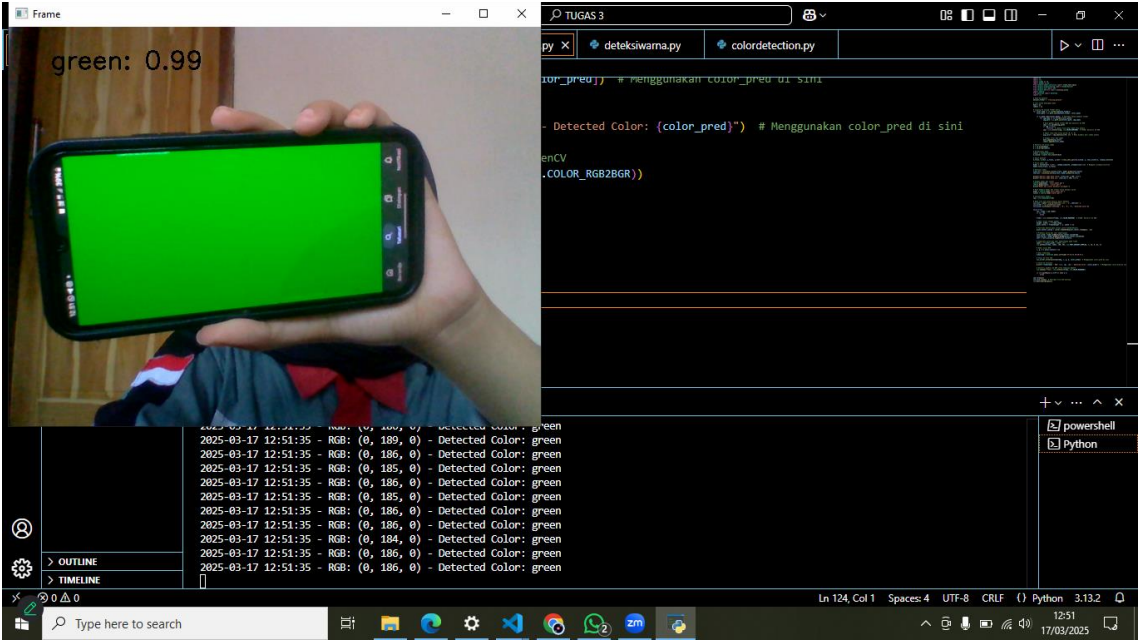
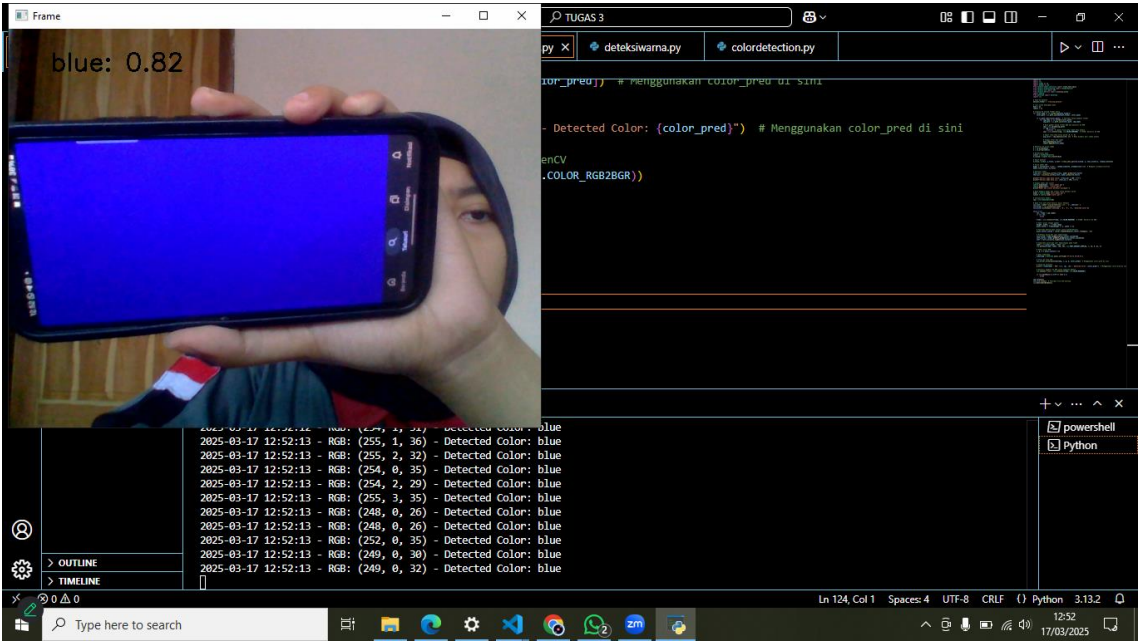
7

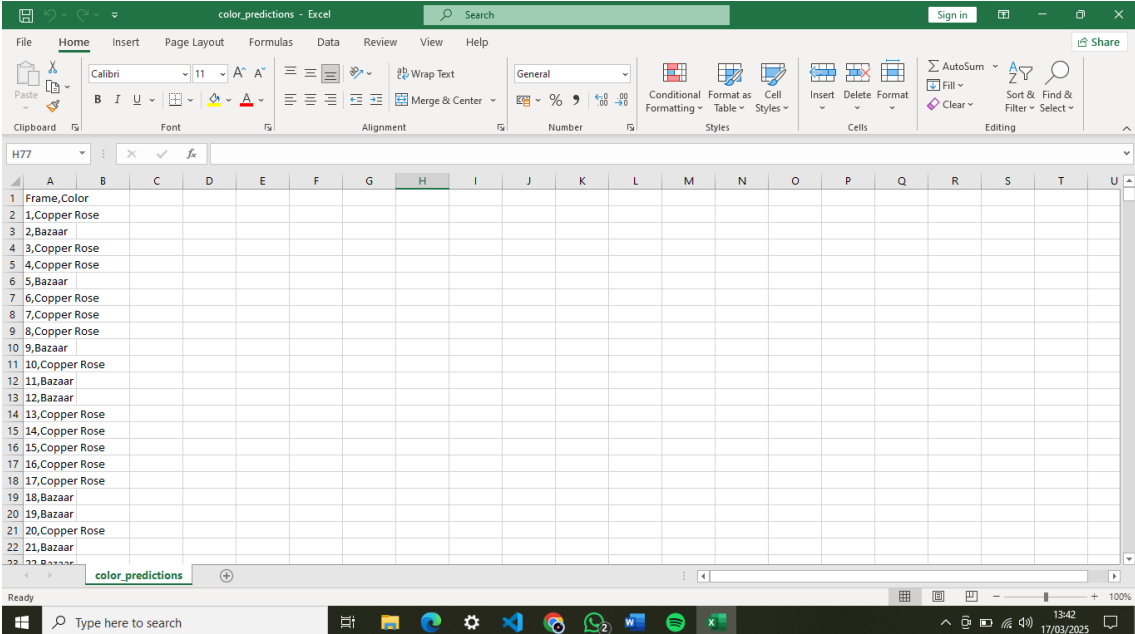
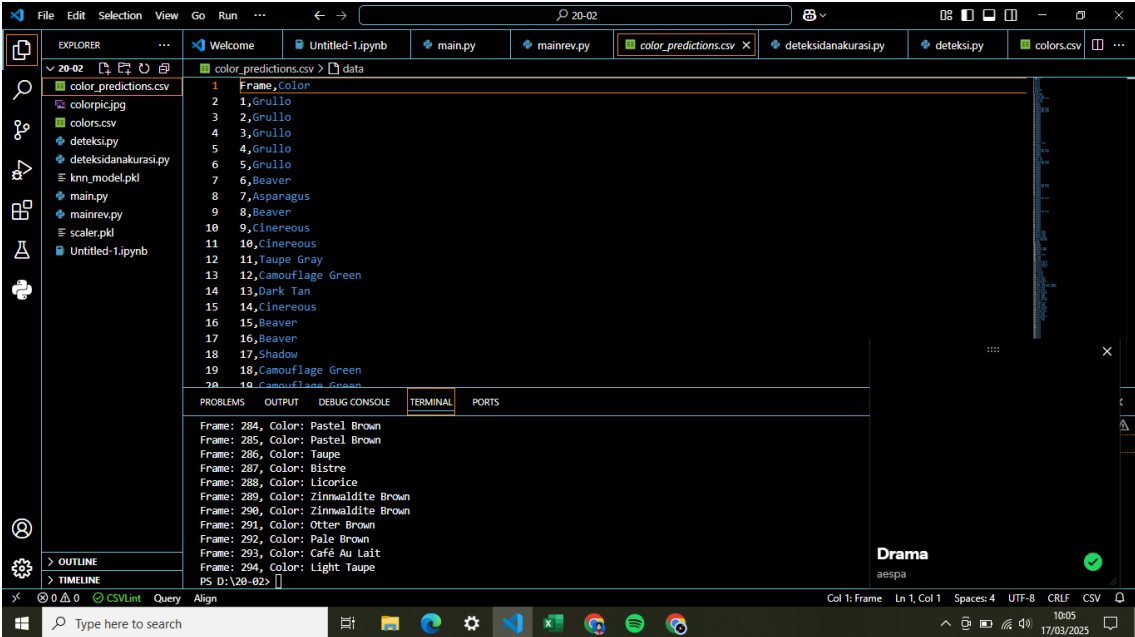
Warna Merah

Output dari program Machine Learning (ML) SVM dengan dataset colors.csv menunjukkan bahwa adanya warna merah terdeteksi.

		
8	Warna Hijau	Output dari program Machine Learning (ML) SVM dengan dataset colors.csv menunjukkan bahwa adanya warna hijau terdeteksi. 
9	Warna Biru	Output dari program Machine Learning (ML) SVM dengan dataset colors.csv menunjukkan bahwa adanya warna biru terdeteksi.

		 The screenshot shows a Windows desktop environment. On the left, a video frame displays a hand holding a smartphone with a blue screen. The text "Detected Color: Medium Blue" is overlaid on the top left of the frame. On the right, a code editor window titled "TUGAS 2" contains Python code for color detection. The code includes imports for cv2, numpy, and sklearn, and a function to detect colors from a video frame. The output of the program is visible in the bottom right corner of the code editor, showing the detected color as "Medium Blue".
10	Warna Merah	Output dari program Machine Learning (ML) SVM dengan dataset training_dataset menunjukkan bahwa adanya warna merah terdeteksi dengan akurasi 0.81.  The screenshot shows a Windows desktop environment. On the left, a video frame displays a hand holding a smartphone with a red screen. The text "red: 0.81" is overlaid on the top left of the frame. On the right, a code editor window titled "TUGAS 3" contains Python code for color detection. The code includes imports for cv2, numpy, and sklearn, and a function to detect colors from a video frame. The output of the program is visible in the bottom right corner of the code editor, showing the detected color as "red" with an accuracy of 0.81.
11	Warna Hijau	Output dari program Machine Learning (ML) SVM dengan dataset training_dataset menunjukkan bahwa adanya warna hijau terdeteksi dengan akurasi 0.99.

		
12	Warna Biru	Output dari program Machine Learning (ML) SVM dengan dataset training_dataset menunjukkan bahwa adanya warna biru terdeteksi dengan akurasi 0.82. 
13	Tabel Data Output KNN	Di bawah ini merupakan data output yang diperoleh dari percobaan yang dilakukan.



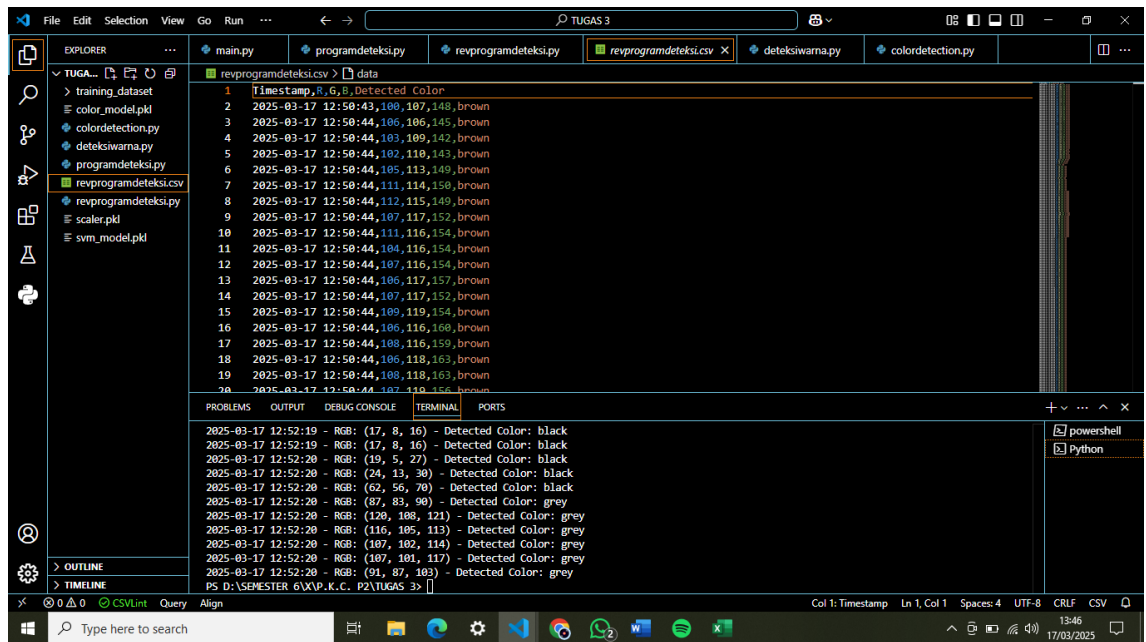
```
1 Timestamp,Color Name,Confidence Score
2 2025-03-17 12:45:00,Licorice,0.84
3 2025-03-17 12:45:00,Licorice,0.85
4 2025-03-17 12:45:00,Licorice,0.81
5 2025-03-17 12:45:00,Licorice,0.83
6 2025-03-17 12:45:00,Licorice,0.85
7 2025-03-17 12:45:00,Licorice,0.83
8 2025-03-17 12:45:00,Licorice,0.86
9 2025-03-17 12:45:00,Licorice,0.88
10 2025-03-17 12:45:00,Licorice,0.85
11 2025-03-17 12:45:00,Licorice,0.83
12 2025-03-17 12:45:00,Licorice,0.85
13 2025-03-17 12:45:00,Licorice,0.83
14 2025-03-17 12:45:00,Licorice,0.86
15 2025-03-17 12:45:00,Licorice,0.82
16 2025-03-17 12:45:01,Licorice,0.87
17 2025-03-17 12:45:01,Licorice,0.86
18 2025-03-17 12:45:01,Licorice,0.84
19 2025-03-17 12:45:01,Licorice,0.81
20 2025-03-17 12:45:01,Licorice,0.84
```

1	Timestamp,Color Name,Confidence Score
2	2025-03-17 12:45:00,Licorice,0.84
3	2025-03-17 12:45:00,Licorice,0.85
4	2025-03-17 12:45:00,Licorice,0.81
5	2025-03-17 12:45:00,Licorice,0.83
6	2025-03-17 12:45:00,Licorice,0.85
7	2025-03-17 12:45:00,Licorice,0.83
8	2025-03-17 12:45:00,Licorice,0.86
9	2025-03-17 12:45:00,Licorice,0.88
10	2025-03-17 12:45:00,Licorice,0.85
11	2025-03-17 12:45:00,Licorice,0.83
12	2025-03-17 12:45:00,Licorice,0.85
13	2025-03-17 12:45:00,Licorice,0.83
14	2025-03-17 12:45:00,Licorice,0.86
15	2025-03-17 12:45:00,Licorice,0.82
16	2025-03-17 12:45:01,Licorice,0.87
17	2025-03-17 12:45:01,Licorice,0.86
18	2025-03-17 12:45:01,Licorice,0.84
19	2025-03-17 12:45:01,Licorice,0.81
20	2025-03-17 12:45:01,Licorice,0.84
21	2025-03-17 12:45:01,Licorice,0.85
22	2025-03-17 12:45:01,Licorice,0.86

14.

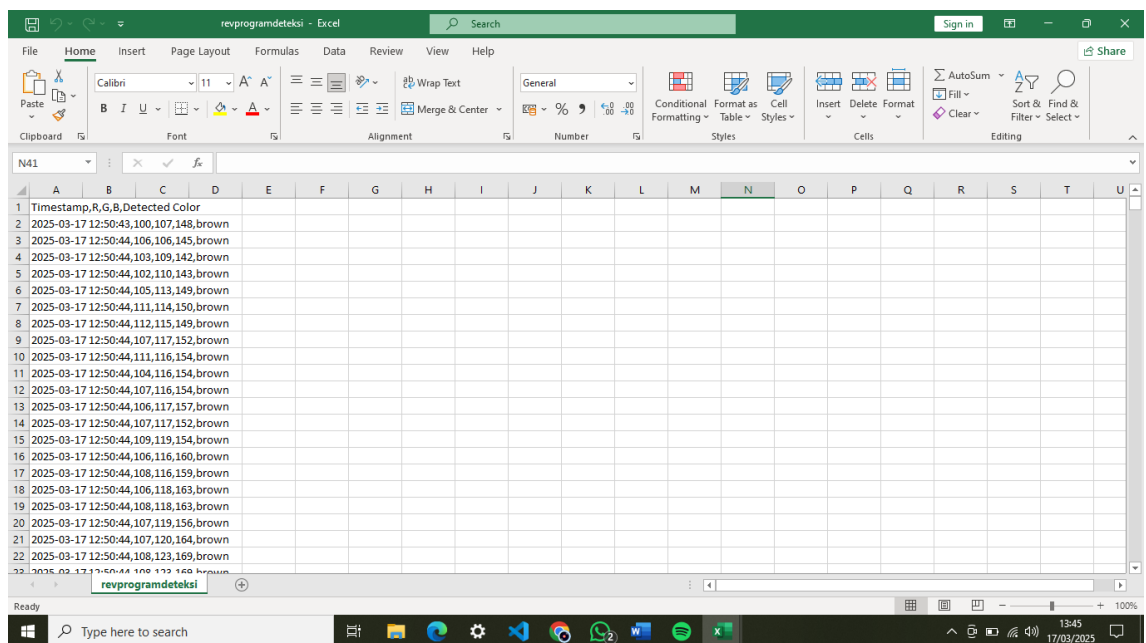
Tabel
Data
Output
SVM

Di bawah ini merupakan data output yang diperoleh dari percobaan yang dilakukan.



```
1 Timestamp,R,G,B,Detected Color
2 2025-03-17 12:50:43,100,107,148,brown
3 2025-03-17 12:50:44,106,106,145,brown
4 2025-03-17 12:50:44,103,109,142,brown
5 2025-03-17 12:50:44,102,110,143,brown
6 2025-03-17 12:50:44,105,113,149,brown
7 2025-03-17 12:50:44,111,114,150,brown
8 2025-03-17 12:50:44,112,115,149,brown
9 2025-03-17 12:50:44,107,117,152,brown
10 2025-03-17 12:50:44,111,116,154,brown
11 2025-03-17 12:50:44,104,116,154,brown
12 2025-03-17 12:50:44,107,116,154,brown
13 2025-03-17 12:50:44,106,117,157,brown
14 2025-03-17 12:50:44,107,117,152,brown
15 2025-03-17 12:50:44,109,119,154,brown
16 2025-03-17 12:50:44,106,116,160,brown
17 2025-03-17 12:50:44,108,116,159,brown
18 2025-03-17 12:50:44,106,118,163,brown
19 2025-03-17 12:50:44,108,118,163,brown
20 2025-03-17 12:50:44,107,119,156,brown
```

```
2025-03-17 12:52:19 - RGB: (17, 8, 16) - Detected Color: black
2025-03-17 12:52:19 - RGB: (17, 8, 16) - Detected Color: black
2025-03-17 12:52:20 - RGB: (19, 5, 27) - Detected Color: black
2025-03-17 12:52:20 - RGB: (24, 13, 30) - Detected Color: black
2025-03-17 12:52:20 - RGB: (62, 56, 70) - Detected Color: black
2025-03-17 12:52:20 - RGB: (87, 83, 90) - Detected Color: grey
2025-03-17 12:52:20 - RGB: (120, 108, 121) - Detected Color: grey
2025-03-17 12:52:20 - RGB: (116, 105, 113) - Detected Color: grey
2025-03-17 12:52:20 - RGB: (107, 101, 117) - Detected Color: grey
2025-03-17 12:52:20 - RGB: (91, 87, 103) - Detected Color: grey
```



Timestamp	R	G	B	Detected Color
2025-03-17 12:50:43	100	107	148	brown
2025-03-17 12:50:44	106	106	145	brown
2025-03-17 12:50:44	103	109	142	brown
2025-03-17 12:50:44	102	110	143	brown
2025-03-17 12:50:44	105	113	149	brown
2025-03-17 12:50:44	111	114	150	brown
2025-03-17 12:50:44	112	115	149	brown
2025-03-17 12:50:44	107	117	152	brown
2025-03-17 12:50:44	111	116	154	brown
2025-03-17 12:50:44	104	116	154	brown
2025-03-17 12:50:44	107	116	154	brown
2025-03-17 12:50:44	106	117	157	brown
2025-03-17 12:50:44	107	117	152	brown
2025-03-17 12:50:44	109	119	154	brown
2025-03-17 12:50:44	106	116	160	brown
2025-03-17 12:50:44	108	116	159	brown
2025-03-17 12:50:44	106	118	163	brown
2025-03-17 12:50:44	108	118	163	brown
2025-03-17 12:50:44	107	119	156	brown

7. Kesimpulan

Berdasarkan percobaan dan analisis data dapat disimpulkan bahwa

- Machine Learning (ML) adalah salah satu percabangan dari kecerdasan buatan yang berfokus pada pengembangan algoritma dan teknik yang memungkinkan untuk belajar dan membuat prediksi atau keputusan berdasarkan data.
- Integrasi ML dalam sistem kendali dapat meningkatkan efisiensi dan efektivitas, meskipun tantangan seperti keterbatasan data dan respons waktu harus diatasi untuk penerapan yang sukses dalam konteks real-time.
- Performa model dalam mendeteksi warna sangat dipengaruhi oleh kualitas dan kuantitas dataset.
- Penambahan jumlah dataset dapat meningkatkan akurasi secara signifikan.

8. Saran

Lebih diperhatikan kembali dalam pemilihan dataset yang digunakan.

9. Daftar Pustaka

- Muttaqin, Arafah M., Jaya A.K., Suryawan M.A., Gustiana Z., Banjarnahor A.R., Bukidz D.P., Hazriani, Simanjuntak M., Saputra N., Fajrillah. 2023. *Implementasi Artificial Intelligence (AI) dalam Kehidupan*. Cetakan ke-1. Yayasan Kita Menulis. Langsa.
- Santoso J.T., 2023. *KECERDASAN BUATAN (Artificial Intelligence)*. Yayasan Prima Agus Teknik Bekerja sama dengan Universitas Sains & Teknologi Komputer (Universitas STEKOM). Semarang.
- Siregar, M. H., & Mulyana, D. I. (2024). Teknologi Artificial Intelligence (AI) Vision Swift dalam Sistem Pemantauan Latihan Bulu Tangkis dengan Algoritma Optical Flow. *Jurnal Indonesia: Manajemen Informatika dan Komunikasi*, 5(3), 3349–3361. <https://doi.org/10.35870/jimik.v5i3.1027>

LAMPIRAN

1. Kode Program Model KNN

```
import cv2

import joblib

import numpy as np

import pandas as pd

import csv

from datetime import datetime


# Muat model KNN dan scaler

knn = joblib.load('knn_model.pkl')

scaler = joblib.load('scaler.pkl')


# Muat dataset warna dari CSV

def load_color_dataset(file_path):

    df = pd.read_csv(file_path)

    return df


# Fungsi untuk mendeteksi warna dan menghitung confidence score

def detect_color(pixel, color_data, max_confidence=1.90):

    pixel_scaled = scaler.transform([pixel])

    distances, indices = knn.kneighbors(pixel_scaled, n_neighbors=1)

    color_pred = knn.predict(pixel_scaled)[0]


# Cek apakah warna ada dalam dataset

if color_pred in color_data['color_name'].values:

    # Hitung confidence score berdasarkan jarak terdekat

    distance = distances[0][0]

    # Gunakan fungsi sigmoid untuk mengubah jarak menjadi skor antara 0 dan 1
```

```

    sigmoid_score = 1 / (1 + np.exp(distance))

    # Scaling confidence score agar mencapai nilai maksimal yang diinginkan
    confidence = max_confidence * sigmoid_score

    return color_pred, confidence

else:

    return "Unknown Color", 0.0


# Inisialisasi kamera
cap = cv2.VideoCapture(0)


# Muat dataset warna
color_data = load_color_dataset('colors.csv')


# Buka file CSV untuk menulis
with open('color_detection_results.csv', 'w', newline='') as csvfile:

    csvwriter = csv.writer(csvfile)

    csvwriter.writerow(['Timestamp', 'Color Name', 'Confidence Score']) # Tulis header


while True:

    ret, frame = cap.read()

    if not ret:

        break


    # Ambil pixel tengah gambar
    height, width, _ = frame.shape

    pixel_center = frame[height // 2, width // 2]


    # Deteksi warna dan hitung confidence score
    color_name, confidence = detect_color(pixel_center, color_data)

```

```
# Format confidence score dengan dua desimal
```

```
confidence_score = f"{confidence:.2f}"
```

```
# Tampilkan warna dan confidence score pada frame
```

```
label = f"{color_name}: {confidence_score}"
```

```
cv2.putText(frame, label, (50, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 0), 2)
```

```
# Dapatkan timestamp saat ini
```

```
timestamp = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
```

```
# Tulis hasil deteksi ke file CSV
```

```
csvwriter.writerow([timestamp, color_name, confidence_score])
```

```
# Tampilkan hasil di terminal
```

```
print(f"Timestamp: {timestamp}, Color: {color_name}, Confidence: {confidence_score}")
```

```
cv2.imshow('Frame', frame)
```

```
if cv2.waitKey(1) & 0xFF == ord('q'):
```

```
    break
```

```
cap.release()
```

```
cv2.destroyAllWindows()
```

2. Kode Program Model SWM

```
import os
```

```
import cv2
```

```
import numpy as np
```

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
import joblib
from datetime import datetime
import csv

# Path ke dataset
dataset_folder = 'training_dataset'

# List untuk menyimpan data
data = []
labels = []

# Looping ke setiap folder warna
for color_name in os.listdir(dataset_folder):
    color_path = os.path.join(dataset_folder, color_name)

    if os.path.isdir(color_path): # Pastikan hanya membaca folder
        for img_name in os.listdir(color_path):
            img_path = os.path.join(color_path, img_name)

            # Baca gambar dalam format BGR dan konversi ke RGB
            img = cv2.imread(img_path)
            if img is None:
                continue # Lewati file yang tidak bisa dibaca
            img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) # FIXED: Konversi ke RGB

            # Ambil rata-rata nilai warna (R, G, B)
```

```
avg_color = img.mean(axis=(0, 1)) # Mean diambil dari semua piksel
```

```
# Simpan fitur dan label
```

```
data.append(avg_color)
```

```
labels.append(color_name)
```

```
# Konversi ke array numpy
```

```
X = np.array(data)
```

```
y = np.array(labels)
```

```
# Normalisasi data
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

```
# Split dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
```

```
# Pilih model SVM
```

```
model = SVC(kernel='linear', random_state=42, probability=True) # Mengatur probability=True
```

```
model.fit(X_train, y_train)
```

```
# Evaluasi model
```

```
train_acc = accuracy_score(y_train, model.predict(X_train))
```

```
test_acc = accuracy_score(y_test, model.predict(X_test))
```

```
print(f"Akurasi pada data latih: {train_acc * 100:.2f}%")
```

```
print(f"Akurasi pada data uji: {test_acc * 100:.2f}%")
```

```
# Simpan model dan scaler
```

```
joblib.dump(model, 'color_model.pkl')
```

```
joblib.dump(scaler, 'scaler.pkl')

print("Model dan scaler berhasil disimpan!")


# Muat kembali model dan scaler untuk deteksi warna
model = joblib.load('color_model.pkl')
scaler = joblib.load('scaler.pkl')


# Inisialisasi kamera
cap = cv2.VideoCapture(0)


# Buka file CSV untuk menulis hasil deteksi
csv_file = open('revprogramdeteksi.csv', 'w', newline='')
csv_writer = csv.writer(csv_file)
csv_writer.writerow(['Timestamp', 'R', 'G', 'B', 'Detected Color'])


while True:
    ret, frame = cap.read()

    if not ret:
        break

    frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB) # FIXED: Konversi ke RGB


    # Ambil pixel tengah gambar
    height, width, _ = frame.shape
    pixel_center = frame[height // 2, width // 2]


    # Pastikan bentuk data sesuai untuk StandardScaler
    pixel_center_scaled = scaler.transform(pixel_center.reshape(1, -1))


    # Prediksi warna dan skor kepercayaan
```

```
color_pred = model.predict(pixel_center_scaled)[0]
conf_scores = model.predict_proba(pixel_center_scaled)[0]
conf = conf_scores[np.argmax(conf_scores)]

# Tampilkan warna dan skor kepercayaan pada frame
label = f"{color_pred}: {conf:.2f}"
cv2.putText(frame, label, (50, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 0), 2)

# Ambil nilai RGB
r, g, b = pixel_center[:-1]

# Buat timestamp
timestamp = datetime.now().strftime('%Y-%m-%d %H:%M:%S')

# Tulis ke file CSV
csv_writer.writerow([timestamp, r, g, b, color_pred]) # Menggunakan color_pred di sini

# Cetak ke terminal
print(f"{timestamp} - RGB: ({r}, {g}, {b}) - Detected Color: {color_pred}") # Menggunakan
color_pred di sini

# Konversi kembali ke BGR untuk tampilan OpenCV
cv2.imshow('Frame', cv2.cvtColor(frame, cv2.COLOR_RGB2BGR))

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
csv_file.close() # Pastikan file CSV ditutup
cv2.destroyAllWindows()
```